Efficiency Unleashed: Reimagining A-to-B Paths in Graphs

Sutejas K[1], K S Ramalakshmi[2], K S Srinivas[3]

[1]Department of Computer Science and Engineering, PES University.
[2]Department of Computer Science and Engineering, PES University.
[3]Department of Computer Science and Engineering(AIML), PES University.

Contributing authors: sutejaskpes@gmail.com; srinivasks@pes.edu; rlsri2305@gmail.com;

Abstract

Efficiently finding the shortest path in a graph is a fundamental problem with diverse applications across numerous domains, from transportation and network routing to social network analysis and bioinformatics. This paper presents a comprehensive approach to improving the efficiency of shortest-path searches in generic graphs. The contributions encompass novel algorithmic enhancements and parallelization techniques to tackle this critical problem. These algorithmic enhancements optimize the traversal of graphs with diverse data types and leverage data encodings to enhance search speed, thus reducing computational overhead and making it feasible to handle large-scale graphs efficiently. To further expedite the shortest path search process, the parallelization technique harnesses the power of modern multi-core processors and distributed computing environments. By efficiently distributing the workload among multiple processing units, this strategy significantly accelerates the state-of-the-art algorithms like Dijkstra's, to explore candidate paths, leading to substantial improvements in query response times. This approach has demonstrated superior performance characteristics in various scenarios and provides an additional layer of optimization. Evaluation of the parallelized approach on benchmark graph datasets has showcased its effectiveness by improving search efficiency and scalability over existing methods like Dijkstra's Algorithm, A* Heuristic Search and the Bellman-Ford Algorithm. By integrating algorithmic enhancements and parallelization, the novel approach opens up new avenues for solving complex shortest-path problems more efficiently with limited computational resources. This paper not only contributes to the field of graph algorithms but also provides a valuable toolkit

for practitioners seeking to optimize shortest-path searches in graph-based data structures.

Keywords: Graphs, Shortest paths, Dijkstra's, A* Heuristic Search, Bellman-Ford, Parallelization, Encoding, Generic graphs, Optimization

1 Introduction

Finding the shortest path in a graph is a fundamental and extensively studied challenge in computer science and mathematics, with a wide array of applications in real-world scenarios. From navigation systems guiding travelers through intricate road networks to optimizing data routing in communication networks, from identifying influential nodes in social networks to unraveling complex biological pathways, the efficient computation of the shortest path is at the core of many critical tasks. Consequently, there is an enduring quest to develop algorithms and techniques that can swiftly and accurately identify the shortest path in a graph, regardless of its size, complexity, or data type.

In this era of big data and complex networks, the need for efficient and scalable solutions for shortest-path searches has never been more pressing. Traditional algorithms like Dijkstra's and A* have long been cornerstones in this field, providing reliable results in a multitude of applications. However, as datasets grow in size and complexity, the computational demands imposed by these algorithms become increasingly burdensome. Moreover, the diversity of graph types encountered in modern applications, including transportation networks, social networks, biological networks, and more, calls for tailored solutions that can adapt to the specific characteristics of each problem domain.

This paper addresses these challenges by presenting a comprehensive approach to improving the efficiency of shortest-path searches in generic graphs. Our approach is multifaceted, encompassing algorithmic enhancements, parallelization techniques, and the integration of state-of-the-art algorithms, all designed to address the unique demands of modern graph-based applications.

This novel approach in this paper introduces algorithmic enhancements that exploit data encodings, leading to faster query response times and efficiently handling a wide range of data types and graph structures.

Recognizing the computational power of modern hardware, the performance of the algorithms is improved through a parallelization framework. By distributing the computation across multiple processing units,the full potential of multicore processors and distributed computing environments is harnessed, significantly accelerating the exploration of candidate paths.

To complement these enhancements, state-of-the-art shortest path algorithms are incorporated into the framework. These algorithms have demonstrated superior performance characteristics in various settings and augment the efficiency of our approach.

Throughout this paper, an in-depth analysis of this novel approach is provided, evaluating its performance on diverse graph datasets and demonstrating its effectiveness in real-world applications. The experimental results showcase substantial improvements in search efficiency and scalability over existing methods, underscoring the practical relevance of these contributions.

The improvised algorithms represent a significant step forward in the domain of shortest-path searches in generic graphs. By combining algorithmic enhancements, parallelization strategies, and the integration of advanced algorithms, it addresses the computational demands of modern applications and paves the way for more efficient and scalable solutions in various domains where efficient pathfinding is critical. This paper discusses about the efforts taken until now to improve graph algorithms, three popular shortest path algorithms, improvisations to the same, comparisons between the results and also discusses the scope for further developments.

2 Related work

At present. several efforts have been taken to analyze and improve the computational efficiency of the shortest path algorithms like Dijkstra's, A* Heuristic Search, and Bellman-Ford.

DongKai Fan[1] has proposed an implementation using an improved storage structure using adjacency lists as against adjacency matrix of topological relations between nodes, to eliminate storage redundancy. Additionally, he has reduced the scale of algorithm searching by restricting the search space, thereby minimizing complexity and improving efficiency. However, with these optimizations, the improved algorithm cannot search genuine shortest paths, it can only give an approximate shortest path, compromising the accuracy. Huang [2] introduced an optimization to improve the Dijkstra Algorithm by introducing the concept of an equivalent path and analyzing its influence factor while drawing a comparison with the traditional approach.

Dexiang Xie[3] approach to shortening Dijkstra's search time by using the heap structure to store the nodes and paths, saved storage space as well. Although the construction of the heap and the heap's linear search will waste some time, this approach still proves efficient. Zhenyu Zhu [4], in the intelligent ship path planning use case, improved Dijkstra's algorithm by reducing the redundant points on the original shortest path given by the classical algorithm. A bio-heuristic intelligent optimization algorithm, called the pheromone factor, from ACO, a probabilistic algorithm to find the optimal path, is applied and the results provide a more practical path.

Chunyu Ju [5] proposed an improvisation over the A* algorithm by considering circular regions around obstacles for local planning as against grid-based A* methods. This led to smoother paths and better obstacle avoidance. Junfeng Yao [6]'s paper in the context of path planning for virtual human motion implemented an improved A* algorithm by using weighted processing of evaluation function. This reduced the searching steps from 200 to 80 and the searching time from 4.359s to 2.823s in the feasible path planning, thus making it more effective and accurate.

Zhonghua Hong [7] introduced optimizations in the A* Star algorithm by implementing a hybrid data structure of the minimum heap and 2D array. This greatly

reduced the time complexity of the algorithm. An optimised search strategy that does not check whether the destination has reached in the initial stage of searching for the global optimal path, improved execution efficiency. Tao Zheng [8] has improved the A* algorithm in the use case of AGV Path Planning, by adding the angle evaluation cost function to the cost function of the A-star algorithm, to find the path with the least inflection point. This reduced the search speed and returned an optimal path.

Goldberg [9] proposed an algorithm that achieves the O(nm) worst-case time complexity as the Bellman-Ford algorithm but in superior practice by using topological scan, involving stacks and DFS computation. However, in the acyclic graphs, it runs in the order of O(m) time. Bannister [10] improvised the Bellman-Ford algorithm by randomly permuting the vertices of the graph, and used that permutation to order the vertices within each pass of the algorithm. The paper provides scope to improve the practical runtime results from the experimental observations.

C. Cheng [11] proposed an implementation of the Bellman-Ford algorithm by avoiding the bouncing effect and the looping problem that occurred in the previous approaches. Chen [12] used the shortest path faster algorithm (SPFA) involving queues to store the nodes and reduce redundancy. The improved algorithm resulted in an optimal path by continuous relaxation operation to the new node.

Through an extensive literature review, it has become evident that previous implementations lacked the identification of redundant points, generalization, and the concept of parallelization, thereby compromising the speed, space efficiency, and accuracy of the results delivered. Furthermore, all previous implementations were designed for integer graphs without generalization. Given the limitations observed, this paper proposes a novel implementation that encodes generic nodes into standard integer nodes and introduces parallelization for enhanced algorithm performance with better time complexity and space complexity.

2.1 Background

2.1.1 Dijkstra's Algorithm

Dijkstra's algorithm [13] is a vital tool in graph theory and optimization to find the shortest path in weighted graphs. It is widely used in areas like network routing and logistics. This algorithm works by systematically picking the node with the lowest known distance from the source and updating distances to its neighbors.

Time Complexity: $O(|E| + |V| \log |V|)$, where E is the number of edges and V is number of nodes in the graph.

Space Complexity: $O(|V|)$, where V is number of nodes in the graph.

2.1.2 A* Heuristic Search Algorithm

A* Heuristic Search algorithm is one of the best and most popular techniques used in path-finding and graph traversals. It serves as a powerful tool for solving optimization problems efficiently. A* combines the benefits of both Dijkstra's algorithm and greedy search. Greedy search is a simple and intuitive algorithm that makes a locally optimal choice at each decision point in the graph to find a global optimum.

Algorithm 1 Dijkstra's Algorithm

```
procedure DijkstraAlgorithm(G, s, t)        ▷ G is the graph, s is the source vertex, t is the target vertex
    dist ← array of distances initialized to ∞
    parent ← array of parent nodes initialized to None
    dist[s] ← 0
    Q ← priority queue of vertices, ordered by dist
    Enqueue(Q, s)
    while Q is not empty do
        u ← Dequeue(Q)
        if u = t then
            return ReconstructPath(parent, t)        ▷ Return the shortest path
        end if
        for each neighbor v of u do
            alt ← dist[u] + weight(u, v)
            if alt < dist[v] then
                dist[v] ← alt
                parent[v] ← u
                Enqueue(Q, v)
            end if
        end for
    end while
    return []                                        ▷ No path from s to t exists
end procedure
procedure ReconstructPath(parent, t)  ▷ Helper function to reconstruct the path
    path ← []
    while t is not None do
        Prepend(path, t)
        t ← parent[t]
    end while
    return path
end procedure
```

This algorithm works by considering the cost to reach a particular state as well as an informed heuristic estimate of the remaining cost to reach the goal. This combination of information allows A* to intelligently prioritize exploring paths that are likely to lead to an optimal solution, making it particularly efficient and adaptable for applications such as route planning in GPS systems, robotics path-finding, and puzzle-solving in video games.

$$f(n) = g(n) + h(n) \tag{1}$$

In this equation 1, *f(n)* is the evaluation function for a node n, *g(n)* is the cost of the path from the start node to node n and *h(n)* is the heuristic estimate of the cost from node n to the goal node. The A* algorithm selects nodes for expansion based on

Algorithm 2 A* Heuristic Search

```
1: procedure AStarSearch(G, s, t)   ▷ G is the graph, s is the source vertex, t is the target vertex
2:     Q ← priority queue of vertices, ordered by f   ▷ Priority queue based on the f value
3:     visited ← set of visited vertices
4:     g ← array of distances initialized to ∞   ▷ Distance from the source
5:     h ← array of heuristic values for each vertex to target t
6:     f ← array of evaluation values initialized to ∞   ▷ f = g + h
7:     g[s] ← 0
8:     f[s] ← h[s]
9:     Enqueue(Q, s)
10:    while Q is not empty do
11:        u ← Dequeue(Q)
12:        if u = t then
13:            return ReconstructPath(s, t, g)   ▷ Return the shortest path
14:        end if
15:        Add u to visited
16:        for each neighbor v of u do
17:            if v is in visited then
18:                continue   ▷ Skip already visited nodes
19:            end if
20:            tempG ← g[u] + weight(u, v)
21:            if tempG < g[v] then
22:                g[v] ← tempG
23:                f[v] ← g[v] + h[v]
24:                Enqueue(Q, v)
25:            end if
26:        end for
27:    end while
28:    return []   ▷ No path from s to t exists
29: end procedure
30: procedure ReconstructPath(s, t, g)   ▷ Helper function to reconstruct the path
31:    path ← []
32:    current ← t
33:    while current ≠ s do
34:        Prepend(path, current)
35:        current ← the neighbor of current with g value that led to current
36:    end while
37:    Prepend(path, s)
38:    return path
39: end procedure
```

the lowest *f(n)* value.

Time Complexity: $O(b^d)$, where b is the branching factor (the average number of successors per state) and d is the number of nodes in the resulting path.

Space Complexity: $O(b^d)$, where b is the branching factor (the average number of successors per state) and d is the number of nodes in the resulting path.

2.1.3 Bellman-Ford Algorithm

The Bellman-Ford algorithm [14] plays a pivotal role in finding the shortest paths in weighted graphs, even in the presence of negative edge weights—a feature that distinguishes it from some other algorithms like Dijkstra's. Bellman-Ford operates by iteratively relaxing edges in the graph, adjusting distance estimates until they converge to the true shortest distances from a source vertex to all other vertices. This attribute makes it particularly suited for scenarios where negative-weight cycles may exist, as it can detect and report their presence. Various optimizations and adaptations have been proposed to enhance its efficiency, making it a versatile tool for tackling real-world problems involving complex interconnected systems.

Time Complexity: $O(|E|*|V|)$, where E is the number of edges and V is number of nodes in the graph.

Space Complexity: $O(|V|)$, where V is number of nodes in the graph.

3 Dataset

In this study, a widely recognized standard benchmark dataset is used [15]. This dataset comprises directed graphs featuring positive edge weights, generated through a Breadth-First Search (BFS) algorithm initiated from a randomly selected point within the New York City benchmark graphs. These New York City benchmark graphs are part of the 9th DIMACS Implementation Challenge, which was conducted in 2005. This dataset comprises 180 distinct graphs, showcasing a wide range of structural characteristics. These graphs vary in size and complexity, with node counts spanning from 108 to 4882 and edge counts ranging from 290 to 14110.

4 Improvements

Optimization in graphs is crucial for efficiently solving complex real-world problems, such as network routing and resource allocation, by finding the most efficient pathways or solutions. Within the three described algorithms, this paper primarily discusses opportunities for parallelization and highlights the runtime optimizations, to improve performance. In order to handle generic graphs, encoding techniques are done to convert graphs with generic nodes to integer nodes.

Algorithm 3 Bellman-Ford Algorithm

```
procedure BellmanFord(G, s)                ▷ G is the graph, s is the source vertex
    dist ← array of distances initialized to ∞
    dist[s] ← 0
    parent ← array of parent nodes initialized to None
    for i ← 1 to |V| − 1 do                                  ▷ Iterate —V— - 1 times
        for each edge (u, v) in G do
            w ← weight of edge (u, v)
            if dist[u] + w < dist[v] then
                dist[v] ← dist[u] + w
                parent[v] ← u
            end if
        end for
    end for
    for each edge (u, v) in G do                  ▷ Check for negative-weight cycles
        w ← weight of edge (u, v)
        if dist[u] + w < dist[v] then
            return "Negative-weight cycle exists"
        end if
    end for
    paths ← array of shortest paths from s to all vertices ▷ Initialize empty paths
array
    for each vertex v in V do
        path ← ReconstructPath(parent, s, v)
        paths[v] ← path
    end for
    return dist, paths                ▷ Shortest distances and paths from source s
end procedure
procedure ReconstructPath(parent, s, v)    ▷ Helper function to reconstruct
the path
    path ← []
    current ← v
    while current ≠ s do
        Prepend(path, current)
        current ← parent[current]
    end while
    Prepend(path, s)
    return path
end procedure
```

4.1 Improved Dijkstra's Algorithm

With reference to the classical Dijkstra's algorithm 1 for all source-all destinations, it is noticed that in line 7, the 'while loop' which performs the traversals can be parallelized. The 'for loop' at line 12 which checks the neighbors, computes new distance, and updates the distance array and parent array, can be parallelized by making line 14, the 'if statement' critical. The path reconstruction for all destinations can also be parallelized and this algorithm can be called for all sources parallely. However, the experimental results show that the 'while loop' in line 7 cannot be parallelized due to its interference with the dequeue operation, giving inaccurate results. The pseudo-code of the improved algorithm with these optimizations is given by 4. In order to obtain all source-all destinations, the call for the algorithm is parallelized to all sources.

4.1.1 Mathematical Formulation

Using adjacency list, with only 1 core,
Time for visiting all vertices: $O(E+V)$, where E is the number of edges and V is number of nodes in the graph
Time for processing one vertex: $O(logV)$
Time for path construction: $O(V^2)$
Time Complexity of Dijkstra's Algorithm with Path reconstruction: $O((V+E)logE+V^2)$

Using adjacency list, for 'p' cores,
Time for visiting all vertices: $O(\frac{V+E}{p})$, where E is the number of edges and V is number of nodes in the graph
Time for Path Reconstruction: $O(\frac{V^2}{p})$
Time Complexity of Parallelized Dijkstra's Algorithm with Path reconstruction: $O(\frac{(V+E)logE}{p}+\frac{V^2}{p})$

4.2 Improved A* Heuristic Search

With reference to the classical A* Heuristic Search Algorithm 2 for a single source-single destination, the 'while loop' in line 10 performing traversal and the 'for loop' in line 16 which checks the neighbors, computes new distance, heuristic and the evaluation value and updates the distance array and parent array, can be parallelized while keeping the 'if statement' in line 21 in the critical section. The experimental results show that the 'while loop' cannot be parallelized due to its interference with the dequeue operation. The pseudo-code of the improved A* heuristic Search can be found at 5.

4.2.1 Mathematical Formulation

Using adjacency list, with only 1 core,
Time for visiting all vertices: $O(E+V)$, where E is the number of edges and V

Algorithm 4 Parallelized Dijkstra's Algorithm

```
1: procedure ParallelizedDijkstra(source)  ▷ source is the source node index
2:     n ← number of nodes in the graph
3:     Initialize paths as an empty array of arrays to store paths
4:     Initialize distance as an array of size n filled with ∞
5:     distance[source] ← 0.0
6:     Initialize parent as an array of size n filled with −1
7:     Initialize visited as an array of size n filled with false
8:     Initialize a priority queue pq as a min-heap of pairs: (distance, vertex)
9:     pq.push({0.0, source})
10:    while not pq is empty do
11:        u ← top vertex from pq          ▷ Critical section to access the top element
12:        if visited[u] then
13:            continue
14:        end if
15:        visited[u] ← true
16:        for each neighbor in adjList[u] do                    ▷ Parallelized loop
17:            v ← neighbor.first
18:            weight ← neighbor.second
19:            if distance[u] + weight < distance[v] then
20:                Critical Section:
21:                distance[v] ← distance[u] + weight
22:                parent[v] ← u
23:                pq.push({distance[v], v})
24:            end if
25:        end for
26:    end while
27:    Path Reconstruction
28:    for i from 0 to n − 1 do                                  ▷ Parallelized loop
29:        if distance[i] = ∞ then
30:            paths[i] ← empty array
31:            continue
32:        end if
33:        Initialize an empty array path
34:        current ← i
35:        while current ≠ −1 do
36:            Insert current at the beginning of path
37:            current ← parent[current]
38:        end while
39:        paths[i] ← path
40:    end for
41:    return paths
42: end procedure
```

Algorithm 5 Parallelized A* Heuristic Search

```
1: procedure ParallelizedAStar(start, goal, heuristic)   ▷ start and goal are node indices, heuristic is a heuristic function
2:     n ← number of nodes in the graph
3:     Initialize distances as an array of size n filled with ∞
4:     distances[start] ← 0.0
5:     Initialize a priority queue Q as a min-heap of pairs: (−distance, vertex)
6:     Q.push({0, start})
7:     Initialize an array parent of size n filled with −1
8:     while not Q is empty do
9:         (current, distance) ← Q.top() ▷ Critical section to access the top element
10:        Q.pop()
11:        if distance > distances[current] then
12:            continue
13:        end if
14:        for each neighbor in adjList[current] do   ▷ Parallelized loop
15:            next ← neighbor.first
16:            weight ← neighbor.second
17:            newDistance ← distance + weight
18:            if newDistance < distances[next] then
19:                Critical Section:
20:                distances[next] ← newDistance
21:                parent[next] ← current
22:                Q.push({−(newDistance+heuristic(dec[current], dec[next])), next})
23:            end if
24:        end for
25:    end while
26:    Initialize an empty array path
27:    current ← goal
28:    while current ≠ −1 do
29:        Insert current at the beginning of path
30:        current ← parent[current]
31:    end while
32:    return path
33: end procedure
```

is number of nodes in the graph
Heuristic Time Complexity: $O(\phi)$
Time for processing one vertex: $O(logV)$
Time Complexity of A* Heuristic Search Algorithm: $O((V+E)*\phi*logV)$

Using adjacency list, for 'p' cores,
Time for visiting all vertices: $O(\frac{V+E}{p})$, where E is the number of edges and V is number of nodes in the graph
Time Complexity of Parallelized A* Heuristic Search: $O(\frac{(V+E)*\phi*logV}{p})$

4.3 Improved Bellman-Ford Algorithm

With reference to the classical Bellman-Ford Algorithm 3 for single source-all destinations, the 'for loop' in line 5 iterates (V-1) times, and the 'for loop' in line 6 which traverses through the edges to remove the negative weights, can be parallelized, by making the 'if statement' in line 8 critical. The 'for loop' in line 14 which checks for negative cycles in the graph can also be parallelized. The path reconstruction for all destinations done by the 'for loop' in line 21 can also be parallelized. Experimentally, the parallelizations resulted in better performance, time complexity and space complexity. The pseudo-code of the improved Bellman-Ford Algorithm 6.

4.3.1 Mathematical Formulation

Using adjacency list, with only 1 core,
Time for traversing all edges (v-1) times: $O(E*V)$, where E is the number of edges and V is number of nodes in the graph
Time for Path Reconstruction: $O(V^2)$
Time Complexity of Bellman-Ford Algorithm: $O((V*E)+V^2)$

Using adjacency list, for 'p' cores,
Time for traversing all edges (v-1) times: $O(\frac{V*E}{p})$, where E is the number of edges and V is number of nodes in the graph
Time for Path Reconstruction: $O(\frac{V^2}{p})$ **Time Complexity of Parallelized Bellman-Ford Algorithm:** $O(\frac{(V*E)}{p}+\frac{V^2}{p})$

5 Empirical Results and Discussion

From the dataset that consisting of directed graphs with positive edge weights that were grown by breadth-first search from a random point in the New York City, 180 to 4882 nodes were chosen to test the enhanced algorithm's performance. The programs were tested on Intel i7 9th generation system and produced the following results for each of the algorithm. The results were drawn in comparison to the top graph libraries

Algorithm 6 Parallelized Bellman-Ford Algorithm

```
1: procedure ParallelizedBellmanFord(source)   ▷ source is the source node index
2:     n ← number of nodes in the graph
3:     Initialize paths as an empty array of arrays to store paths
4:     Initialize distance as an array of size n filled with ∞
5:     Initialize parent as an array of size n filled with −1
6:     distance[source] ← 0.0
7:     for i from 0 to n − 1 do
8:         for each node u from 0 to n − 1 do                    ▷ Parallelized loop
9:             for each neighbor in adjList[u] do                ▷ Parallelized loop
10:                v ← neighbor.first
11:                weight ← neighbor.second
12:                if distance[u] + weight < distance[v] then
13:                    Critical Section:
14:                    distance[v] ← distance[u] + weight
15:                    parent[v] ← u
16:                end if
17:            end for
18:        end for
19:    end for
20:    Check for negative cycles:
21:    local_negative_cycle ← false
22:    for each node u from 0 to n − 1 do                        ▷ Parallelized loop
23:        for each neighbor in adjList[u] do                    ▷ Parallelized loop
24:            v ← neighbor.first
25:            weight ← neighbor.second
26:            if distance[u] + weight < distance[v] then
27:                local_negative_cycle ← true
28:            end if
29:        end for
30:    end for
31:    if local_negative_cycle then          ▷ Critical section for error reporting
32:        throw runtime error("Negative cycle detected.")
33:    end if
34:    Path Reconstruction:
35:    for i from 0 to n − 1 do                                  ▷ Parallelized loop
36:        if distance[i] = ∞ then
37:            paths[i] ← empty array
38:            continue
39:        end if
40:        Initialize an empty array path
41:        current ← i
42:        while current ≠ −1 do
43:            Insert current at the beginning of path
44:            current ← parent[current]
45:        end while
46:        paths[i] ← path
47:    end for
48:    return paths
49: end procedure
```

currently in use, namely C++ based Boost graph Library and Python-based NetworkX and iGraph.

Table 1 Algorithm and Complete Runtime in (μ seconds) for A* Heuristic Search

		New Method		BGL		NetworkX		iGraph	
Nodes	Edges	AR	CR	AR	CR	AR	CR	AR	CR
108	290	0	959	570	2337	1000	46989	998	335000
136	368	0	1002	744	2349	1007	2036	999	336000
171	470	0	1956	1080	3257	1041	3034	1000	337001
214	584	0	1992	1266	4056	1998	3998	1000	383010
268	726	0	2004	1307	4612	2000	4000	1001	313004
335	910	0	2016	1631	5197	2001	14000	1001	287900
419	1140	0	2999	1969	6197	2033	8040	1002	264000
524	1406	0	3998	2233	6574	3002	6991	1004	313001
655	1758	0	4996	2563	38402	3010	8009	1009	335966
819	2226	0	5959	2859	39538	4002	18993	1031	306042
1024	2824	960	8984	3272	28104	4993	15990	1035	286045
1280	3548	999	9999	3579	12757	5001	15998	1040	288051
1600	4442	1000	12961	4951	21097	6999	20000	1042	344040
2000	5596	1972	19002	6675	24215	8000	27001	2000	317984
2500	6946	1999	22001	8532	28555	10008	44010	2002	284999
3125	8646	2960	29957	15688	34547	13007	44996	2002	292000
3906	10736	3000	30999	20280	46679	19988	59999	3000	342004
4882	14110	4000	42966	26990	184239	26999	75031	3002	367999

Table 2 Algorithm and Complete Runtime in (μ seconds) for Bellman-Ford's Algorithm

		New Method		BGL		NetworkX		iGraph	
Nodes	Edges	AR	CR	AR	CR	AR	CR	AR	CR
108	290	0	988	570	2337	23012	24011	4004	149994
136	368	985	1959	744	2349	30958	31994	5001	144001
171	470	1001	3013	1080	3257	37009	39010	6999	158960
214	584	1034	4049	1266	4056	45001	47043	10998	141998
268	726	2000	5000	1307	4612	77013	79006	18991	166002
335	910	2995	4995	1631	5197	125965	128000	27000	152999
419	1140	3988	6988	1969	6197	198008	201001	42008	216994
524	1406	6988	11998	2233	6574	323994	328002	65000	209000
655	1758	6999	11998	2563	38402	510975	515002	139990	276998
819	2226	9995	15991	2859	39538	1227957	1235991	167953	335967
1024	2824	11965	20000	3272	28104	2061041	2070035	267988	424998
1280	3548	19049	33055	3579	12757	2319994	2331006	395038	546005
1600	4442	25958	38984	4951	21097	3591965	3604954	617999	770993
2000	5596	35990	50958	6675	24215	5440000	5455005	987958	1147958
2500	6946	49999	67944	8532	28555	11524000	11541998	1663007	1824005
3125	8646	77007	100979	15688	34547	15080665	15104522	2414970	2575978
3906	10736	109999	140030	20280	46679	22564001	22591039	4016006	4188005
4882	14110	141964	178958	26990	184239	45273714	45325712	6616001	6791002

The mentioned tables draw comparisons between the libraries based on their algorithmic and complete runtime. Algorithmic runtime is the time taken to run the

Table 3 Algorithm and Complete Runtime in (*milli* seconds) for Dijkstra's Algorithm

		New Method		BGL		NetworkX		iGraph	
108	290	69	70	42	44	41	42	529	658
136	368	134	136	56	366	60	72	742	883
171	470	227	228	97	117	84	93	1453	1592
214	584	381	383	150	174	149	158	2816	2979
268	726	871	875	234	237	249	259	5833	5985
335	910	1284	1288	438	442	330	340	10359	10508
419	1140	2243	2246	558	563	507	521	19696	19841
524	1406	3903	3907	857	862	901	913	39890	40041
655	1758	6778	6785	1349	1423	1431	1445	78523	78705
819	2226	11831	11841	2100	2328	2380	2395	156813	156949
1024	2824	20548	20561	3625	3688	4430	4448	275239	275386
1280	3548	34726	34735	5210	5252	6810	6830	497456	497598
1600	4442	61412	61425	8452	8496	10441	10462	1144850	1144993
2000	5596	108312	108329	12119	12256	17500	17527	1144993	1956648

algorithm, given the constructed graph, and complete runtime is the total time taken for the execution of the program from the construction of the graph to the end of the algorithm.

It can be inferred from tables 5, 5, 5 that as the cost of computation increases with the increase in the graph size, the parallelized method performs significantly better and is greatly optimised in comparison to the traditional approaches implemented as a part of the graph libraries.

6 Conclusion and Future Scope

This paper highlights the importance of optimizing graph theory algorithms, particularly the shortest path search, showcasing their relevance in solving real-world challenges like routing and resource management through reduced computational complexity and enhanced practicality. This paper also provides a solid foundation for extending its implementation to state-of-the-art algorithms like Johnson's algorithm, while also opening the door to research opportunities in other graph-related areas, including Minimum Spanning Trees and Centrality Measures.

The research primarily focuses on two key enhancements: utilizing hashmap encoding to convert generic graphs into integer graphs and implementing parallelization techniques in algorithms like Dijkstra's, A* Heuristic Search, and Bellman-Ford Algorithms. The empirical results provide strong evidence that these implementations can significantly boost the efficiency of these algorithms.

References

[1] Fan, D., Shi, P.: Improvement of dijkstra's algorithm and its application in route planning. In: 2010 Seventh International Conference on Fuzzy Systems and Knowledge Discovery, vol. 4, pp. 1901–1904 (2010). IEEE

[2] Huang, Y.: Research on the improvement of dijkstra algorithm in the shortest path calculation. In: 2017 4th International Conference on Machinery, Materials

and Computer (MACMC 2017), pp. 745–749 (2018). Atlantis Press

[3] Xie, D., Zhu, H., Yan, L., Yuan, S., Zhang, J.: An improved dijkstra algorithm in gis application. In: World Automation Congress 2012, pp. 167–169 (2012). IEEE

[4] Zhu, Z., Li, L., Wu, W., Jiao, Y.: Application of improved dijkstra algorithm in intelligent ship path planning. In: 2021 33rd Chinese Control and Decision Conference (CCDC), pp. 4926–4931 (2021). IEEE

[5] Ju, C., Luo, Q., Yan, X.: Path planning using an improved a-star algorithm. In: 2020 11th International Conference on Prognostics and System Health Management (PHM-2020 Jinan), pp. 23–26 (2020). IEEE

[6] Yao, J., Lin, C., Xie, X., Wang, A.J., Hung, C.-C.: Path planning for virtual human motion using improved a* star algorithm. In: 2010 Seventh International Conference on Information Technology: New Generations, pp. 1154–1158 (2010). IEEE

[7] Hong, Z., Sun, P., Tong, X., Pan, H., Zhou, R., Zhang, Y., Han, Y., Wang, J., Yang, S., Xu, L.: Improved a-star algorithm for long-distance off-road path planning using terrain data map. ISPRS International Journal of Geo-Information **10**(11), 785 (2021)

[8] Zheng, T., Xu, Y., Zheng, D.: Agv path planning based on improved a-star algorithm. In: 2019 IEEE 3rd Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC), pp. 1534–1538 (2019). IEEE

[9] Busato, F., Bombieri, N.: An efficient implementation of the bellman-ford algorithm for kepler gpu architectures. IEEE Transactions on Parallel and Distributed Systems **27**(8), 2222–2233 (2015)

[10] Bannister, M.J., Eppstein, D.: Randomized Speedup of the Bellman–Ford Algorithm, pp. 41–47

[11] Cheng, C., Riley, R., Kumar, S.P., Garcia-Luna-Aceves, J.J.: A loop-free extended bellman-ford routing protocol without bouncing effect. ACM SIGCOMM Computer Communication Review **19**(4), 224–236 (1989)

[12] Chen, H., Suh, H.-J.: An improved bellman-ford algorithm based on spfa. The Journal of the Korea institute of electronic communication sciences **7**(4), 721–726 (2012)

[13] Dijkstra, E.W.: A note on two problems in connexion with graphs. Numerische mathematik **1**(1), 269–271 (1959)

[14] Walden, D.: The bellman-ford algorithm and "distributed bellman-ford (2008)

[15] Planken, L.R.L., Weerdt, M., Krogt, R.P.J.R.: Benchmark instances for experiments in "Computing all-pairs shortest paths by leveraging low treewidth" - New York (Figure 6). TU Delft - Delft University of Technology; Faculty EEMCS; Algorithmics group (2011). https://doi.org/10.4121/UUID:4B6D7F56-F09B-4507-8E1A-3078D6E0266A . https://data.4tu.nl/articles/_/12693512/1